

# Guia Completo: Integração Banco de Dados ↔ Front-end

## Sistema de Clínica Médica — Java + PostgreSQL + HTML/JS

---

### Entendendo o que vai acontecer

Antes de começar, entenda o caminho que os dados percorrem:

```
Usuário clica no botão
  → JavaScript faz fetch() para o Servlet
    → Servlet recebe a requisição
      → Servlet chama o DAO
        → DAO executa SQL no PostgreSQL
          → Resultado volta pelo mesmo caminho
            → JavaScript atualiza a tela
```

Você já tem as peças prontas:

- ☒ Banco de dados (script SQL)
  - ☒ Classes Model (Paciente.java, Agendamento.java etc.)
  - ☒ Classes DAO (PacienteDAO.java, AgendamentoDAO.java etc.)
  - ☒ Front-end (index.html + script.js com dados simulados)
  - ☒ Faltam: Servlets, Gson, web.xml e script.js atualizado
- 

## PARTE 1 — Preparação do Ambiente

### Passo 1.1 — Baixar as bibliotecas necessárias

Você precisa de 3 arquivos .jar dentro da pasta `ClinicaMedica/lib/`:

#### 1. PostgreSQL JDBC Driver (já mencionado no README)

- Acesse: <https://jdbc.postgresql.org/download/>
- Baixe: `postgresql-42.6.0.jar`
- Coloque em: `ClinicaMedica/lib/`

#### 2. Gson (converte objetos Java ↔ JSON)

- Acesse: <https://search.maven.org/artifact/com.google.code.gson/gson/2.10.1/jar>
- Clique em "jar" para baixar
- Coloque em: `ClinicaMedica/lib/`

### 3. Servlet API (para criar os Servlets)

- Se usar Tomcat, o arquivo já está em: `$TOMCAT_HOME/lib/servlet-api.jar`
- Você não copia ele para o lib/ do projeto, só usa na compilação

Ao final, sua pasta `lib/` deve ter:

```
ClinicaMedica/lib/  
├─ postgresql-42.6.0.jar  
└─ gson-2.10.1.jar
```

#### Passo 1.2 — Verificar a senha do banco

Abra o arquivo `src/com/ucsal/clinica/util/DatabaseConfig.java` e edite:

```
java  
  
private static final String DB_URL = "jdbc:postgresql://localhost:5432/clinica";  
private static final String DB_USER = "postgres";           // seu usuário  
private static final String DB_PASSWORD = "sua_senha";       // sua senha real aqui
```

#### Passo 1.3 — Verificar se o banco está rodando

Abra o terminal e teste:

```
bash  
  
psql -U postgres -d clinica -c "SELECT COUNT(*) FROM clinica.paciente;"
```

Se aparecer um número (mesmo que zero), o banco está funcionando. Se der erro, o PostgreSQL não está rodando — inicie pelo pgAdmin ou pelo terminal com `pg_ctl start`.

---

## PARTE 2 — Criando o web.xml

O `web.xml` é o "mapa" da aplicação web. Ele diz ao Tomcat quais servlets existem e em qual endereço cada um responde.

Crie o arquivo em: `ClinicaMedica/web/WEB-INF/web.xml`

```
xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
  version="3.1">

  <display-name>Clínica Médica</display-name>

  <!-- Servlet de Pacientes -->
  <servlet>
    <servlet-name>PacienteServlet</servlet-name>
    <servlet-class>com.ucsal.clinica.servlet.PacienteServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>PacienteServlet</servlet-name>
    <url-pattern>/pacientes</url-pattern>
  </servlet-mapping>

  <!-- Servlet de Agendamentos -->
  <servlet>
    <servlet-name>AgendamentoServlet</servlet-name>
    <servlet-class>com.ucsal.clinica.servlet.AgendamentoServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>AgendamentoServlet</servlet-name>
    <url-pattern>/agendamentos</url-pattern>
  </servlet-mapping>

  <!-- Servlet de Profissionais -->
  <servlet>
    <servlet-name>ProfissionalServlet</servlet-name>
    <servlet-class>com.ucsal.clinica.servlet.ProfissionalServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>ProfissionalServlet</servlet-name>
    <url-pattern>/profissionais</url-pattern>
  </servlet-mapping>

  <!-- Servlet de Consultas -->
  <servlet>
    <servlet-name>ConsultaServlet</servlet-name>
    <servlet-class>com.ucsal.clinica.servlet.ConsultaServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>ConsultaServlet</servlet-name>
```

```
        <url-pattern>/consultas</url-pattern>
    </servlet-mapping>

    <!-- Servlet do Dashboard (estatísticas) -->
    <servlet>
        <servlet-name>DashboardServlet</servlet-name>
        <servlet-class>com.ucsal.clinica.servlet.DashboardServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>DashboardServlet</servlet-name>
        <url-pattern>/dashboard</url-pattern>
    </servlet-mapping>

    <!-- Página inicial -->
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>

</web-app>
```

**Por que isso é necessário?** Sem o web.xml, o Tomcat não sabe que `/pacientes` deve ser respondido pelo `PacienteServlet`. É como um catálogo de endereços.

---

## PARTE 3 — Criando um objeto auxiliar para resposta

Antes dos Servlets, crie uma classe auxiliar que facilita enviar respostas JSON padronizadas.

**Crie o arquivo:** `src/com/ucsal/clinica/util/RespostaJson.java`

```
java
```

```
package com.ucsal.clinica.util;

// Essa classe representa a resposta padrão que o Servlet envia ao JavaScript
public class RespostaJson {
    private boolean sucesso;
    private String mensagem;
    private Object dados; // pode ser qualquer coisa: lista, objeto, etc.

    // Construtor para sucesso com dados
    public RespostaJson(boolean sucesso, String mensagem, Object dados) {
        this.sucesso = sucesso;
        this.mensagem = mensagem;
        this.dados = dados;
    }

    // Construtor simples (sem dados, só sucesso/erro)
    public RespostaJson(boolean sucesso, String mensagem) {
        this.sucesso = sucesso;
        this.mensagem = mensagem;
    }

    public boolean isSucesso() { return sucesso; }
    public String getMensagem() { return mensagem; }
    public Object getDados() { return dados; }
}
```

## PARTE 4 — Criando os Servlets

Os Servlets são a "ponte" entre o JavaScript e o banco. Cada um responde a um endereço específico.

### Passo 4.1 — PacienteServlet

**Crie o arquivo:** `src/com/ucsal/clinica/servlet/PacienteServlet.java`

```
java
```

```
package com.ucsal.clinica.servlet;

import com.google.gson.Gson;
import com.ucsal.clinica.dao.PacienteDAO;
import com.ucsal.clinica.dao.UsuarioDAO;
import com.ucsal.clinica.model.Paciente;
import com.ucsal.clinica.model.Usuario;
import com.ucsal.clinica.util.RespostaJson;

import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;

@WebServlet("/pacientes")
public class PacienteServlet extends HttpServlet {

    private PacienteDAO pacienteDAO = new PacienteDAO();
    private UsuarioDAO usuarioDAO = new UsuarioDAO();
    private Gson gson = new Gson();

    // GET /pacientes → retorna lista de pacientes
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {

        // Configura o tipo de resposta como JSON
        resp.setContentType("application/json");
        resp.setCharacterEncoding("UTF-8");

        // Permite que o JavaScript de outra origem acesse (CORS)
        resp.setHeader("Access-Control-Allow-Origin", "*");

        try {
            List<Paciente> pacientes = pacienteDAO.listarTodos();
            // Converte a lista para JSON e envia
            resp.getWriter().write(gson.toJson(pacientes));
        } catch (Exception e) {
            resp.setStatus(500); // Erro interno
            RespostaJson erro = new RespostaJson(false, "Erro ao buscar pacientes: ");
            resp.getWriter().write(gson.toJson(erro));
        }
    }
}
```

```

    }
}

// POST /pacientes → salva um novo paciente
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws IOException {

    resp.setContentType("application/json");
    resp.setCharacterEncoding("UTF-8");
    resp.setHeader("Access-Control-Allow-Origin", "*");

    try {
        // Lê o JSON que veio do JavaScript e converte para um objeto
        // O JSON esperado é: { "nome": "...", "cpf": "...", "email": "...", ...
        PacienteRequest dados = gson.fromJson(req.getReader(), PacienteRequest.class);

        // Passo 1: Salva o usuário (todo paciente é também um usuário)
        Usuario usuario = new Usuario();
        usuario.setNomeCompleto(dados.nome);
        usuario.setEmail(dados.email);
        usuario.setSenhaHash("123456"); // Em produção: gerar hash de verdade
        usuario.setAtivo(true);
        usuario.setIdPerfil(5); // 5 = perfil Paciente (conforme seu banco)
        usuario.setDataCriacao(LocalDateTime.now());
        usuario.setDataAtualizacao(LocalDateTime.now());

        boolean usuarioCriado = usuarioDAO.inserir(usuario);

        if (!usuarioCriado) {
            resp.setStatus(500);
            resp.getWriter().write(gson.toJson(new RespostaJson(false, "Erro ao criar usuário")));
            return;
        }

        // Passo 2: Salva o paciente vinculado ao usuário
        Paciente paciente = new Paciente();
        paciente.setIdUsuario(usuario.getIdUsuario()); // usa o ID gerado acima
        paciente.setCpf(dados.cpf.replaceAll("[^0-9]", "")); // remove pontos e traço
        paciente.setDataNascimento(LocalDate.parse(dados.dataNascimento));
        paciente.setSexo(dados.sexo.charAt(0));
        paciente.setAtivo(true);
        paciente.setDataCriacao(LocalDateTime.now());

        boolean pacienteCriado = pacienteDAO.inserir(paciente);

        if (pacienteCriado) {
            // Retorna o JSON do paciente criado
            PacienteResponse resposta = new PacienteResponse(paciente);
            resp.setStatus(201);
            resp.getWriter().write(gson.toJson(resposta));
        }
    }
}

```

```

        resp.setStatus(201); // 201 = Created (criado com sucesso)
        resp.getWriter().write(gson.toJson(
            new RespostaJson(true, "Paciente cadastrado com sucesso!", pacie
        ));
    } else {
        resp.setStatus(500);
        resp.getWriter().write(gson.toJson(new RespostaJson(false, "Erro ao

    }

} catch (Exception e) {
    resp.setStatus(500);
    resp.getWriter().write(gson.toJson(
        new RespostaJson(false, "Erro interno: " + e.getMessage())
    ));
}

}

// Classe interna que representa o JSON recebido do JavaScript
// Os campos precisam ter exatamente os mesmos nomes que o JS envia
private static class PacienteRequest {
    String nome;
    String cpf;
    String email;
    String dataNascimento; // formato: "1990-05-15"
    String sexo;           // "M" ou "F"
    String telefone;
}
}

```

## Passo 4.2 — AgendamentoServlet

**Crie o arquivo:** `src/com/ucsal/clinica/servlet/AgendamentoServlet.java`

```
java
```



```
package com.ucsal.clinica.servlet;

import com.google.gson.Gson;
import com.ucsal.clinica.dao.AgendamentoDAO;
import com.ucsal.clinica.model.Agendamento;
import com.ucsal.clinica.util.RespostaJson;

import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.List;

@WebServlet("/agendamentos")
public class AgendamentoServlet extends HttpServlet {

    private AgendamentoDAO agendamentoDAO = new AgendamentoDAO();
    private Gson gson = new Gson();

    // GET /agendamentos → retorna lista de agendamentos
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {

        resp.setContentType("application/json");
        resp.setCharacterEncoding("UTF-8");
        resp.setHeader("Access-Control-Allow-Origin", "*");

        try {
            // Verifica se foi pedido filtro por data
            String dataFiltro = req.getParameter("data"); // ex: ?data=2024-01-15

            List<Agendamento> agendamentos;
            if (dataFiltro != null && !dataFiltro.isEmpty()) {
                agendamentos = agendamentoDAO.buscarPorData(LocalDate.parse(dataFiltro));
            } else {
                agendamentos = agendamentoDAO.listarTodos();
            }

            resp.getWriter().write(gson.toJson(agendamentos));

        } catch (Exception e) {
```

```

        resp.setStatus(500);
        resp.getWriter().write(gson.toJson(
            new RespostaJson(false, "Erro ao buscar agendamentos: " + e.getMessage()
        ));
    }
}

// POST /agendamentos → cria novo agendamento
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws IOException {

    resp.setContentType("application/json");
    resp.setCharacterEncoding("UTF-8");
    resp.setHeader("Access-Control-Allow-Origin", "*");

    try {
        AgendamentoRequest dados = gson.fromJson(req.getReader(), AgendamentoReq

        Agendamento agendamento = new Agendamento();
        agendamento.setIdPaciente(dados.idPaciente);
        agendamento.setIdProfissional(dados.idProfissional);
        agendamento.setIdAtendente(1); // Em produção: pegar da sessão do usuário
        agendamento.setDataConsulta(LocalDate.parse(dados.dataConsulta));
        agendamento.setHoraConsulta(LocalTime.parse(dados.horaConsulta));
        agendamento.setMotivoConsulta(dados.motivoConsulta);
        agendamento.setDataAgendamento(LocalDateTime.now());
        agendamento.setIdStatus(1); // 1 = "Agendado"
        agendamento.setDataCriacao(LocalDateTime.now());
        agendamento.setDataAtualizacao(LocalDateTime.now());

        boolean criado = agendamentoDAO.inserir(agendamento);

        if (criado) {
            resp.setStatus(201);
            resp.getWriter().write(gson.toJson(
                new RespostaJson(true, "Agendamento realizado com sucesso!", age
            ));
        } else {
            resp.setStatus(500);
            resp.getWriter().write(gson.toJson(new RespostaJson(false, "Erro ao

        }

    } catch (Exception e) {
        resp.setStatus(500);
        resp.getWriter().write(gson.toJson(
            new RespostaJson(false, "Erro interno: " + e.getMessage())

```

```
        ));  
    }  
}  
  
private static class AgendamentoRequest {  
    int idPaciente;  
    int idProfissional;  
    String dataConsulta; // "2024-01-15"  
    String horaConsulta; // "09:00"  
    String motivoConsulta;  
}  
}
```

### Passo 4.3 — DashboardServlet

Crie o arquivo: `src/com/ucsal/clinica/servlet/DashboardServlet.java`

java

```

package com.ucsal.clinica.servlet;

import com.google.gson.Gson;
import com.ucsal.clinica.dao.AgendamentoDAO;
import com.ucsal.clinica.dao.PacienteDAO;
import com.ucsal.clinica.util.RespostaJson;

import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

@WebServlet("/dashboard")
public class DashboardServlet extends HttpServlet {

    private PacienteDAO pacienteDAO = new PacienteDAO();
    private AgendamentoDAO agendamentoDAO = new AgendamentoDAO();
    private Gson gson = new Gson();

    // GET /dashboard → retorna os números do dashboard
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {

        resp.setContentType("application/json");
        resp.setCharacterEncoding("UTF-8");
        resp.setHeader("Access-Control-Allow-Origin", "*");

        try {
            // Monta um objeto com as estatísticas
            Map<String, Object> stats = new HashMap<>();
            stats.put("totalPacientes", pacienteDAO.listarTodos().size());
            stats.put("agendamentosHoje", agendamentoDAO.contarAgendamentosHoje());
            // Adicione mais contadores conforme necessário

            resp.getWriter().write(gson.toJson(stats));

        } catch (Exception e) {
            resp.setStatus(500);
            resp.getWriter().write(gson.toJson(
                new RespostaJson(false, "Erro ao carregar dashboard: " + e.getMessage()
            ));
        }
    }
}

```

```
}  
}
```

---

## PARTE 5 — Adicionando métodos que faltam nos DAOs

O `AgendamentoDAO` precisa de alguns métodos que ainda não existem. Adicione-os:

No arquivo `AgendamentoDAO.java`, adicione:

```
java
```

```

// Busca agendamentos de uma data específica
public List<Agendamento> buscarPorData(LocalDate data) throws SQLException {
    String sql = "SELECT a.*, " +
        "u_pac.nome_completo AS nome_paciente, " +
        "u_prof.nome_completo AS nome_profissional, " +
        "sa.descricao AS status_descricao " +
        "FROM clinica.agendamento a " +
        "JOIN clinica.paciente p ON a.id_paciente = p.id_paciente " +
        "JOIN clinica.usuario u_pac ON p.id_usuario = u_pac.id_usuario " +
        "JOIN clinica.profissional_saude ps ON a.id_profissional = ps.id_pr
        "JOIN clinica.usuario u_prof ON ps.id_usuario = u_prof.id_usuario "
        "JOIN clinica.status_agendamento sa ON a.id_status = sa.id_status "
        "WHERE a.data_consulta = ? " +
        "ORDER BY a.hora_consulta";

    List<Agendamento> agendamentos = new ArrayList<>();

    try (Connection conn = DatabaseConfig.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setDate(1, Date.valueOf(data));

        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                agendamentos.add(mapResultSetParaAgendamento(rs));
            }
        }
    }

    return agendamentos;
}

// Conta quantos agendamentos existem para hoje
public int contarAgendamentosHoje() throws SQLException {
    String sql = "SELECT COUNT(*) FROM clinica.agendamento WHERE data_consulta = CUR

    try (Connection conn = DatabaseConfig.getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        if (rs.next()) {
            return rs.getInt(1);
        }
    }

    return 0;
}

```

```
// Também precisa de listarTodos() – adicione se não existir:
public List<Agendamento> listarTodos() throws SQLException {
    String sql = "SELECT * FROM clinica.agendamento ORDER BY data_consulta DESC, hora_consulta ASC";
    List<Agendamento> agendamentos = new ArrayList<>();

    try (Connection conn = DatabaseConfig.getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        while (rs.next()) {
            agendamentos.add(mapResultSetParaAgendamento(rs));
        }
    }
    return agendamentos;
}
```

## PARTE 6 — Atualizando o script.js

Agora substitua os dados simulados do `script.js` por chamadas reais à API.

Substitua a função `loadDashboard()`:

```
javascript

async function loadDashboard() {
    try {
        const response = await fetch('/ClinicaMedica/dashboard');
        const stats = await response.json();

        document.getElementById('agendamentosHoje').textContent = stats.agendamentos;
        document.getElementById('totalPacientes').textContent = stats.totalPacientes;
        document.getElementById('totalProfissionais').textContent = stats.totalProfissionais;
        document.getElementById('examesPendentes').textContent = stats.examesPendentes;

    } catch (erro) {
        console.error('Erro ao carregar dashboard:', erro);
        showAlert('Erro ao carregar o dashboard', 'error');
    }

    loadProximosAgendamentos();
}
```

Substitua `loadPacientesTable()`:





## Substitua o listener do formPaciente:

javascript

```
const formPaciente = document.getElementById('formPaciente');
if (formPaciente) {
  formPaciente.addEventListener('submit', async function(e) {
    e.preventDefault();

    // Pega os dados do formulário
    const dados = {
      nome: document.getElementById('nomePaciente').value,
      cpf: document.getElementById('cpfPaciente').value,
      dataNascimento: document.getElementById('dataNascimento').value,
      sexo: document.getElementById('sexoPaciente').value,
      telefone: document.getElementById('telefonePaciente').value,
      email: document.getElementById('emailPaciente').value
    };

    try {
      const response = await fetch('/ClinicaMedica/pacientes', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json' // diz que estamos enviando
        },
        body: JSON.stringify(dados) // converte o objeto JS para texto JSON
      });

      const resultado = await response.json();

      if (resultado.sucesso) {
        showAlert('Paciente cadastrado com sucesso!', 'success');
        this.reset(); // limpa o formulário
        loadPacientes(); // recarrega a tabela
      } else {
        showAlert('Erro: ' + resultado.mensagem, 'error');
      }
    } catch (erro) {
      console.error('Erro:', erro);
      showAlert('Erro ao cadastrar paciente', 'error');
    }
  });
}
```

---

## PARTE 7 — Compilando o projeto

### Passo 7.1 — Estrutura de pastas antes de compilar

Verifique se as pastas existem:

```
bash

mkdir -p ClinicaMedica/bin
mkdir -p ClinicaMedica/web/WEB-INF/lib
mkdir -p ClinicaMedica/web/WEB-INF/classes
```

### Passo 7.2 — Compilar (Linux/Mac)

```
bash

cd ClinicaMedica

# Compila tudo de uma vez
javac -d bin \
  -cp "lib/postgresql-42.6.0.jar:lib/gson-2.10.1.jar:$TOMCAT_HOME/lib/servlet-api.jar" \
  src/com/ucsal/clinica/model/*.java \
  src/com/ucsal/clinica/util/*.java \
  src/com/ucsal/clinica/dao/*.java \
  src/com/ucsal/clinica/servlet/*.java
```

### Passo 7.3 — Compilar (Windows)

```
bat

cd ClinicaMedica

javac -d bin ^
  -cp "lib/postgresql-42.6.0.jar;lib/gson-2.10.1.jar;C:\tomcat\lib\servlet-api.jar" ^
  src\com\ucsal\clinica\model\*.java ^
  src\com\ucsal\clinica\util\*.java ^
  src\com\ucsal\clinica\dao\*.java ^
  src\com\ucsal\clinica\servlet\*.java
```

Se aparecer algum erro de compilação, ele vai indicar o arquivo e a linha. Os erros mais comuns são:

- `cannot find symbol` → faltou um import ou um método não existe
- `package does not exist` → o .jar não está no classpath (-cp)

---

## PARTE 8 — Deploy no Tomcat

### Passo 8.1 — Copiar arquivos

```
bash

# Cria a estrutura de deploy
mkdir -p dist/ClinicaMedica/WEB-INF/lib
mkdir -p dist/ClinicaMedica/WEB-INF/classes

# Copia as classes compiladas
cp -r bin/* dist/ClinicaMedica/WEB-INF/classes/

# Copia as bibliotecas
cp lib/postgresql-42.6.0.jar dist/ClinicaMedica/WEB-INF/lib/
cp lib/gson-2.10.1.jar      dist/ClinicaMedica/WEB-INF/lib/

# Copia o web.xml
cp web/WEB-INF/web.xml dist/ClinicaMedica/WEB-INF/

# Copia os arquivos do front-end
cp web/html/index.html dist/ClinicaMedica/
cp web/css/style.css    dist/ClinicaMedica/
cp web/js/script.js     dist/ClinicaMedica/

# Faz o deploy no Tomcat (ajuste o caminho do seu Tomcat)
cp -r dist/ClinicaMedica $TOMCAT_HOME/webapps/
```

### Passo 8.2 — Iniciar o Tomcat

```
bash

$TOMCAT_HOME/bin/startup.sh    # Linux/Mac
$TOMCAT_HOME\bin\startup.bat   # Windows
```

### Passo 8.3 — Acessar a aplicação

Abra no navegador:

---

## PARTE 9 — Testando e depurando

### Como testar se o servlet funciona

Antes de abrir o HTML, teste o servlet direto no navegador:

`http://localhost:8080/ClinicaMedica/pacientes`

Se aparecer `[]` (lista vazia) ou uma lista de pacientes em JSON, o servlet está funcionando. Se aparecer erro 404, o `web.xml` está errado ou o deploy não foi feito corretamente. Se aparecer erro 500, há um problema no Java (verifique os logs do Tomcat).

### Onde ver os erros do Tomcat

Os logs ficam em `$TOMCAT_HOME/logs/catalina.out`. Quando algo der errado, esse arquivo mostra a causa exata.

### Erros mais comuns e soluções

| Erro                                      | Causa                                        | Solução                                                                          |
|-------------------------------------------|----------------------------------------------|----------------------------------------------------------------------------------|
| 404 no servlet                            | URL errada ou <code>web.xml</code> incorreto | Confirme o <code>url-pattern</code> no <code>web.xml</code>                      |
| 500 no servlet                            | Exceção no Java                              | Veja o <code>catalina.out</code>                                                 |
| Dados não aparecem                        | JS não acessa o servlet                      | Abra o console do navegador (F12) e veja a aba Network                           |
| <code>ClassNotFoundException: Gson</code> | Gson não está no WEB-INF/lib                 | Copie o <code>gson.jar</code> para WEB-INF/lib                                   |
| <code>Connection refused</code>           | Banco não está rodando                       | Inicie o PostgreSQL                                                              |
| CPF duplicado                             | Constraint do banco                          | Trate a exceção <code>SQLIntegrityConstraintViolationException</code> no Servlet |

### Resumo da ordem de execução

- ✓ Baixar `postgresql-42.6.0.jar` e `gson-2.10.1.jar` → colocar em `lib/`
- ✓ Editar `DatabaseConfig.java` com sua senha
- ✓ Criar `web/WEB-INF/web.xml`
- ✓ Criar `src/com/ucsal/clinica/util/RespostaJson.java`
- ✓ Criar `PacienteServlet.java`, `AgendamentoServlet.java`, `DashboardServlet.java`

6. ☒ Adicionar métodos `listarTodos()` e `contarAgendamentosHoje()` no `AgendamentoDAO`
7. ☒ Atualizar as funções no `script.js`
8. ☒ Compilar com `javac`
9. ☒ Copiar arquivos para o Tomcat
10. ☒ Iniciar o Tomcat e testar no navegador